Grounded Literature Review Generation with Multi-Agent LLMs and Citation-Relation Learning

Introduction to Artificial Intelligence (CSS_2203)
<SEMESTER> SEMESTER MINI PROJECT REPORT
Submitted
to

MANIPAL ACADEMY OF HIGHER EDUCATION

School of Computer Engineering
SoCE – AI, <Section>

Submitted by

Siddharth Ray
Reg. No. 235890416
<STUDENT NAME>
<reg no.>
<STUDENT NAME>
<reg no.>

Under the guidance of

Dr. Simi V.R.
Assistant Professor- Senior Scale
School of Computer Engineering
Manipal Institute of Technology, Bengaluru



MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

October 2026

MANIPAL INSTITUTE OF TECHNOLOGY
BENGALURU
(A constituent unit of MAHE, Manipal)

ABSTRACT

A large language model can write a literature review that reads well and still describe papers it has not understood. The sentences look right, the citations are real, and only someone who reads the cited papers can tell that a claim does not belong to the paper it is attached to. This project builds a system that makes that failure visible. It is a team of six language-model agents, built with the CrewAI framework, that reads a set of papers and writes a related-work section in which every sentence can be traced back to a quoted line from a specific paper. A separate checking model decides whether each sentence really follows from the lines behind it, and anything that fails is rewritten once and then marked rather than hidden.

The training data comes from the papers themselves. The related-work section of any published paper is a short literature review that an expert has already written, and the papers it cites are known. Hiding that section and keeping only the cited papers' abstracts gives a task with a correct answer, at no annotation cost. From 200 arXiv papers in computer-science language and information retrieval, the pipeline produced 15,704 citation sentences and 80 usable test examples. 656 of those citation sentences were labelled with one of seven relation types, such as whether a paper is being used, compared against or disagreed with. A classifier fine-tuned from SciBERT scores 0.13 macro-F1 on a hand-checked set, against 0.23 for a simple word-counting baseline, and the machine labels it learns from agree with the hand-checked ones at a kappa of 0.27.

The checker finds 59% of extracted claims fully supported by the line they quote. The gap between quoting correctly and being supported is the problem this project is about: a model usually copies the quote accurately and still writes a claim the quote does not prove. The full comparison between systems had not been run when this report was built, and Chapter 4 says exactly which parts were and were not measured. Every number in this report is read from a file the code wrote, and every figure is drawn from those files by a single script.

Table of Contents

Table of Contents

			Page No
Abstract			ii
List of Tables			1
List of Figures			1
Chapter 1		**INTRODUCTION**	2
	1.1	Overview of the project work	2
	1.2	Motivation and background of the study	2
	1.3	Problem statement and relevance	3
	1.4	Objectives	3
Chapter 2		**BACKGROUND THEORY / LITERATURE REVIEW**	4
	2.1	Summary of existing studies / Comparative analysis of various approaches	6
	2.2	Key findings, trends, and research gaps	6
	2.4	Conclusion	6
Chapter 3		**METHODOLOGY**	8
	3.1	Methodology with diagrams	8
	3.2	System requirements	13
Chapter 4		**ANALYSIS AND DISCUSSION**	14
Chapter 5		**CONCLUSION AND FUTURE SCOPE**	21
	5.1	Conclusion	21
	5.2	Suggestions for future directions	21
References			23

LIST OF TABLES

Table No	Table Title	Page No
Table 1	How different approaches compare	6
Table 2	What the system was run on, and the exact versions used for the results in Chapter 4	13
Table 3	Stage 1 mining yield at pilot scale	14
Table 4	Dataset statistics for the mined arXiv benchmark and the external Multi-XScience benchmark	15
Table 5	Citation-relation classification on the 200-context adjudicated gold set	16
Table 6	Failure categories counted over every cached generation in the evaluation run	19

LIST OF FIGURES

Page numbers are Word fields; press Ctrl+A then F9 to refresh them after editing.

Figure No	Figure Title	Page No
Figure 1	How the system fits together	8
Figure 2	Mining papers from arXiv	8
Figure 3	The ten steps, the point where a human approves the paper list, and the single rewrite loop	11
Figure 4	Mining yield: papers surviving each step, as counts and as a percentage of the submissions listed	14
Figure 5	Distribution of LLM-assigned citation relations over the labelled pool	15
Figure 6	Confusion matrices on the adjudicated gold set, for the LLM labeller and for the fine-tuned classifier	16
Figure 7	Accuracy and macro-F1 of each relation-classification system on the adjudicated gold set	16
Figure 8	Verifier calibration	17
Figure 9	Verifier verdicts over extracted claims, by extraction run	17
Figure 10	The worked example in the console	18

CHAPTER 1
INTRODUCTION

1.1 Overview of the project work

This project builds a tool that writes the “related work” part of a research report, and checks its own writing before handing it over. You give it a topic or a few starting papers. It finds related papers, reads each one, pulls out the main claims, and keeps a short quoted line from the paper next to every claim as proof. It then compares the papers against each other to see where they agree, where they disagree, and which combinations nobody has tried. Finally it writes the section, and every sentence it writes carries a list of the claims it was built from.

The work is divided between six agents, each with a narrow job: a Librarian that finds papers, an Extractor that pulls out claims, a Verifier that checks them, a Synthesist that compares papers, a Gap Analyst that looks for missing work, and a Writer. The limits matter as much as the jobs: the Extractor cannot search, so it can only use the text it is given, and the Writer cannot check its own output, because a model asked to grade its own work tends to agree with itself. Only the Verifier may run the checking model.

Two parts of the system are trained rather than prompted. The first is a classifier that reads a sentence containing a citation and decides what the sentence is doing: using the cited work, comparing against it, or disagreeing with it. The second is a small model trained to copy the extraction step, so the same work can be done on a laptop instead of through a paid interface.

The same idea supplies the test set. Every published paper already contains a short literature review written by an expert — its related-work section — and the papers it cites are listed. Hiding that section and giving a system only the cited papers' abstracts creates a test with a correct answer that nobody had to write by hand.

1.2 Motivation and background of the study

Language models write fluent summaries that are often wrong in ways fluency hides. Maynez and colleagues [1] found that a large share of summaries from strong models contain statements the source document never made, and that the usual scoring method, ROUGE, barely distinguishes those summaries from correct ones. A later survey [2] found the same pattern across many text-generation tasks.

For a literature review this is a serious problem rather than an annoying one. A made-up reference is easy to catch, because the paper does not exist. A real reference attached to a claim that paper never made is much harder to catch, because every part of the sentence checks out except the link between them. A reader would have to open the cited paper to notice, which is exactly the work the tool was supposed to save.

Researchers in summarisation responded by checking whether a summary follows from its source, instead of comparing it against a reference answer. One line of work generates questions from the summary and checks that the source gives the same answers [3]. Another trains a classifier on deliberately corrupted summaries [4]. A third uses natural language inference, the task of deciding whether one sentence follows from another, applied one sentence at a time [5]. This project follows the third approach and adds one requirement: the system must say which line each sentence came from, so there is something definite to check against.

Meanwhile it has become easy to split a task across several model-driven agents with their own roles and tools [6], [7], and to have a model criticise and revise its own output [8], [9].

What is usually missing is an honest answer to whether all that extra machinery is worth its cost. This project measures that directly, with the same model and settings for every system compared, and reports the answer whichever way it comes out.

1.3 Problem statement and relevance

The problem is this: given a research topic and the papers that matter to it, produce a literature review in which every sentence can be traced to a specific line in a specific paper, together with a map of where those papers agree and disagree and a list of gaps — and do it within the limits of a free API account and one free cloud GPU.

Three smaller problems follow. First, there is no ready-made dataset that says what a citing sentence is claiming about the paper it cites, so that signal has to be mined from published papers. Second, the checking cannot be done by asking the same model again, because it will usually agree with itself; it needs a separate model doing a different task. Third, the result has to be compared against something a human wrote, or the claim that the output is well-grounded cannot be tested at all.

The practical case is simple. Any active research area now produces more papers than a student can read before writing about them, and the tools that offer to help are the same tools that invent things. A slower system whose every sentence can be checked against a quoted line is more useful than a fast one whose output has to be re-read against the sources anyway.

1.4 Objectives

Each objective below names the measurement that decides whether it was met. Chapter 4 reports those measurements.

1. Mine related-work sections and citation sentences from arXiv source files, and report how many papers survive each step as a percentage of those listed.

2. Define seven citation relation types with a written labelling guide, label sentences with a language model, and measure how far those labels can be trusted by comparing them against a set labelled independently by hand.

3. Fine-tune SciBERT on those labels and beat three baselines — always guessing the most common class, a word-counting classifier, and asking a language model with no training — on macro-F1.

4. Extract claims that quote their evidence word for word, and check each claim twice: that the quote really appears in the source, and that the claim follows from the quote. Set the thresholds for both checks by testing them on a hand-checked sample rather than guessing.

5. Build a map of agreement and disagreement between papers and a table of which method families have been tried on which datasets, and have agents argue both for and against every disagreement and every gap they report.

6. Train a small local model to copy the extraction step, using only extractions that passed the checker, and compare the two on output validity, claim support, speed and cost.

7. Compare four systems and one cut-down version on papers published after every tested model's training cut-off, reporting overlap with the human text, how many cited papers are discussed, how often a sentence is attached to the right paper, how many sentences survive checking, and what each system cost to run.

CHAPTER 2
BACKGROUND THEORY / LITERATURE REVIEW

Table 1 at the end of this chapter summarises how the approaches below compare. This chapter covers the five areas this project draws on: summarising several documents at once, working out what a citation is doing, checking whether generated text is actually supported, teams of language-model agents, and tools that help write literature reviews. Every paper named here was checked against OpenAlex, Semantic Scholar or the arXiv listing before it was cited; the record of those checks is in results/references_verified.json.

Summarising scientific papers

Summarising a single document improved in two steps: models that could copy rare words straight out of the source [10], then large pre-trained models adapted to the task, of which BART [11] and PEGASUS [12] remain the usual points of comparison. Scientific papers broke the assumption that a document fits inside the model at once: Cohan and colleagues [13] handled longer documents by attending to section structure and released the arXiv and PubMed datasets, while XSum [14] pushed the other way, asking for a single sentence that cannot be produced by copying.

Writing a related-work section is older than today's models. Abu-Ra'ed and colleagues [15] treated it as summarising the cited papers; Wang and colleagues [16] attended to both the citing paper's content and the cited abstracts; and Luu and colleagues [17] reframed it as explaining the relationship between two papers, the closest existing idea to the relation classifier used here.

Multi-XScience [18] turned the task into a standard benchmark: each example pairs a paper's related-work paragraph with the abstracts it cites, which is exactly the setup described in Chapter 3. This project uses it as a second test set so results can be compared against a published dataset and not only against our own mining. The large-scale parsed full text both rely on comes from S2ORC [19]. More recently Zhang and colleagues [20] found that instruction-tuned models match or beat specially trained summarisers in human judgement while the automatic scores disagree — a direct warning against reading ROUGE on its own, and one the evaluation here takes seriously.

Working out what a citation is doing

Citations differ, and the differences are what a review must express. Early work looked at citation sentiment and at how much surrounding text was needed [21], [22]. Two datasets made the task standard: ACL-ARC, used by Jurgens and colleagues [23] to trace how a field changes, with six categories; and SciCite [24], which traded detail for size with three. SciBERT [25], a BERT trained on scientific text, is the usual starting point for both and is the starting point here.

The seven categories used in this project are finer than SciCite's and cut differently from ACL-ARC's, for one reason that matters later. The system needs to tell a neutral comparison apart from an actual disagreement. Running someone else's method as a baseline and reporting that you scored higher is a measurement. Saying their conclusion does not hold is a disagreement, and only the second is evidence that two papers conflict. SciCite's "result" class and ACL-ARC's "compare or contrast" class both merge the two. Section 3.1 sets out how those datasets were mapped onto our categories and what that mapping loses; because it loses something, mapped rows are used only for extra training and never for scoring.

Checking whether generated text is supported

The finding behind this project is that fluent summaries are often unsupported and that overlap scores do not notice [1]. Three families of automatic check followed. FactCC [4] trains a classifier on summaries that were deliberately broken in known ways. QAGS [3] asks questions about the summary and checks whether the source gives the same answers. SummaC [5] revisits natural language inference — deciding whether one sentence follows from another — and shows it works well once applied sentence by sentence rather than to whole documents. That last design is the one the checker here copies.

Judging those checks needed human annotation: SummEval [26] showed how weakly standard scores track human opinion, and Goyal and Durrett [27] found that different model families fail differently. BERTScore [28] is used here as a meaning-based alternative to ROUGE, not as a support check, since it compares against a reference rather than the source.

Fact-checking supplies the other half: FEVER [29] established retrieving evidence then testing whether it supports a claim, and SciFact [30] moved that into science, where claims are hedged. DeBERTa [31] is the model design used here as the checker. The hallucination survey [2] supplies this report's wording: a claim is unsupported when the evidence does not establish it, which is weaker and more checkable than calling it false.

Teams of language-model agents

Asking a model to work through a problem in steps was the first sign that breaking a task up helps [32], and tools let a model act outside its own text [33]. Several agents working together followed: CAMEL [6] showed that agents given roles can carry a task further than one prompt, and generative agents [7] showed memory and planning over long runs. Self-criticism was formalised as Self-Refine [8] and, with a separate critic, as Reflexion [9].

CrewAI belongs to this family: agents with a role, a goal and tools; tasks whose output can be forced into a fixed shape; and flows that connect steps with explicit order and shared state. Two features carry real weight here. A fixed output shape is what lets the Extractor be checked by code rather than read by eye. Named flow steps let the human approval point be a real stop, and let an interrupted run carry on — which matters when everything must fit inside a free account's limits.

The important lesson from this literature, for this project, is that a model checking its own work is using the same model that produced it, and its judgement is tied to the error that caused the problem. The checker here is therefore not an agent asked to be sceptical. It is a separate, smaller model doing a different job on a well-defined pair of texts, and the agent whose role is called Verifier is allowed to do nothing except call it.

Tools that help write literature reviews

Retrieval-augmented generation [34] with meaning-based search [35] is the standard way to tie a model's output to documents, and the practical tools build on it. PaperQA [36] forces answers through retrieved passages and reports less invention; STORM [37] researches a topic from several points of view, builds an outline and writes a long cited article; Sušnjak and colleagues [38] fine-tune subject-specific models for research synthesis.

Two things separate the system built here. The first is how finely it is accountable. STORM and PaperQA attach citations at the level of a passage or a paragraph; this system records, for every sentence, the exact claim identifiers it used, then checks that sentence against those claims' quoted lines and no others. The second is that two parts are trained rather than prompted: the citation-relation classifier, trained on sentences mined from real papers, and the extractor, compressed into a small open model [39] using QLoRA [40] with the checker deciding which examples it may learn from.

2.1 Summary of existing studies / works / Comparative analysis of various approaches

Table 1: How different approaches compare. The third column is what a generated statement can be traced back to, and the fourth is what, if anything, checks that the trace is real.

Approach	Example work	What a statement traces back to	What checks it	Citation types	Trained part
Pre-trained summariser	BART [11], PEGASUS [12]	nothing specific	nothing	no	whole model
Related-work generator	Wang et al. [16], Multi-XScience [18]	a cited abstract	nothing	no	a task-specific model
Citation-type classifier	SciCite [24], ACL-ARC [23]	the citing sentence	not applicable	3 or 6	SciBERT
Support checker	SummaC [5], QAGS [3]	a summary sentence	inference or questions	no	an off-the-shelf model
Retrieval-based agent	PaperQA [36], STORM [37]	a retrieved passage	retrieval only	no	none, prompted
Self-criticising loop	Self-Refine [8], Reflexion [9]	the whole output	the same model	no	none, prompted
This project	Chapters 3–4	a quoted line per claim; claim ids per sentence	a separate model, thresholds fitted on a sample	7, trained	relation classifier and extractor

2.2 Key findings, trends, and research gaps

Four trends run through the work above. Checking has moved from comparing against a reference answer towards testing whether the output follows from the source. What a statement is tied to has got finer, from whole documents to single passages. Citation types have been standardised, but at a granularity chosen to make annotation cheap rather than the labels useful. And orchestration has moved from single prompts to teams of agents, usually with self-criticism as the quality mechanism.

Three gaps follow, and they are what this project addresses. First, the support-checking work tests a summary against one source document, while the review-writing work generates from many sources and is scored against a reference text; almost nothing checks a many-source review against the sources it cites, sentence by sentence. Second, citation labels are used to study how fields evolve rather than to drive the writing: we know of no system that treats a predicted disagreement as evidence that two papers conflict and then asks an agent to explain it. Third, the cost of running a team of agents is rarely reported next to its quality, so the literature does not say whether the extra calls are worth making.

A fourth point is about method rather than subject. Several of the agent papers report improvements without separating training and test material by date, on tasks whose reference answers may already be inside the model being tested. Section 3.1 and docs/leakage.md therefore record exactly which publication dates were used here and why the test window starts after every tested model's training cut-off.

2.4 Conclusion

All the pieces this project needs already exist and are well studied on their own: pre-trained summarisers, citation classifiers, inference-based support checks, agent frameworks and retrieval. What has not been put together is a pipeline where the citation relation is learned from real related-work sections, every written sentence names the evidence behind it, a separate model judges that evidence using thresholds fitted on real examples, and the whole thing is then measured against human-written sections on papers the models cannot have seen, with its running cost reported next to its quality. Chapter 3 describes how that was built and Chapter 4 reports what it achieved, including the places where it did not beat a single well-written prompt.

CHAPTER 3
METHODOLOGY

3.1 Methodology with diagrams

The system is one Python package used in two ways: command-line scripts for running locally, and six Colab notebooks for the larger runs. The notebooks install the package and call the same functions, so there is only one copy of the logic and the two cannot drift apart. Figure 1 shows the parts and how information moves between them.

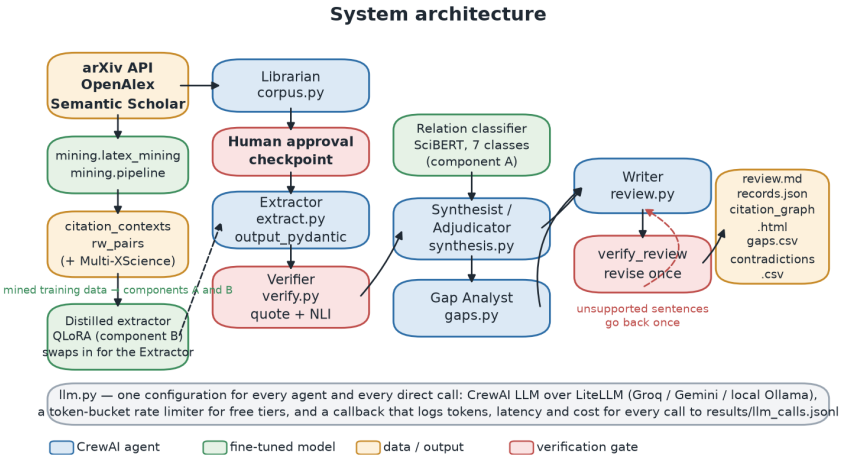


Figure 1: How the system fits together. Blue boxes are agents, green boxes are trained models, red boxes are checks, and orange boxes are data or output. One model setting serves every agent and records what every call cost.

3.1.1 Where the training data comes from

The system needs to learn what a citing sentence is claiming about the paper it cites, and no ready-made dataset provides that at the level of detail needed. The solution is to take it from published papers. A related-work section is a short literature review an expert already wrote, and it names the papers it cites, so mining those sections gives two things at once: citing sentences whose meaning is visible from the text around them, and pairs of (cited abstracts, written section) that form a test set with a correct answer. Figure 2 shows the steps.

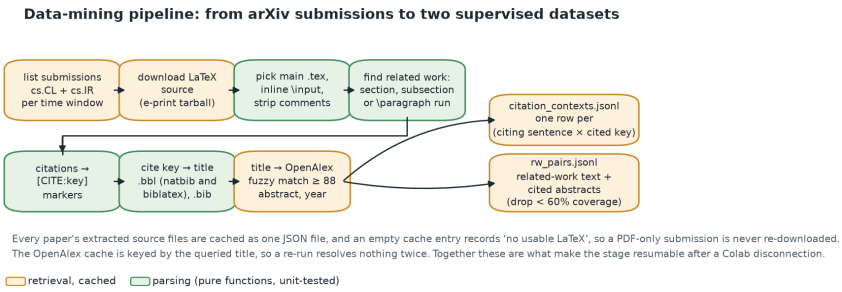


Figure 2: Mining papers from arXiv. Each paper's source files are stored once, and a paper with no usable source is recorded as such so it is never downloaded twice. That is what lets the step be stopped and restarted.

arXiv distributes most papers as LaTeX source, which is readable text rather than a PDF. The parser handles the awkward cases, each found in a real paper during testing and each now covered by a test: several candidate main files, related work written as a sub-section or as a run of unnamed paragraphs, two bibliography formats, a bibliography written into the main file, and the citation commands used by different publishers. A heading alone is not accepted as proof: a section titled “Related Work” must contain at least one citation and a vaguer heading

such as “Background” at least two, or a section explaining standard theory would be mined as though it were a survey.

Cited papers then have to be identified. Where the bibliography gives a DOI or an arXiv number, that number identifies the paper exactly, and those are looked up in batches. Only where no identifier exists does the system fall back to searching by title, which is both slower and less certain; a title match is accepted only above a similarity threshold, so the wrong paper is rarely attached. Papers for which fewer than 60% of the cited works could be found with an abstract are dropped from the test set, because the task is only fair if the system can see most of what the original author saw.

3.1.2 Telling citation types apart

Seven labels are used: background, uses-method, uses-data, extends, compares-against, supports and contrasts. The split that matters most is between the last three. Running another paper's method as a baseline and reporting a better score is compares-against, which is a measurement. Saying your results agree with theirs is supports. Saying their conclusion does not hold is contrasts, and only that last one is evidence that two papers actually conflict.

Because real sentences often fit more than one label, the rules are applied in a fixed order: disagreement beats agreement, which beats extending, which beats comparing, which beats using data, which beats using a method, with background as the fallback. The definitions and this order live in one file, so the instructions given to the labelling model, the guide a human labeller follows and the training code cannot disagree about what a label means.

Two published datasets, SciCite [24] and ACL-ARC [23], are mapped onto these seven labels to provide extra training examples. The mapping loses information and the report says where: SciCite's “method” class does not separate using a method from improving it, and its “result” class covers comparing, agreeing and disagreeing alike; ACL-ARC's “uses” does not separate method from data, and its “compare or contrast” merges neutral comparison with real disagreement. Because of that, mapped examples are used only to help training and never to score anything.

The classifier is SciBERT, fine-tuned to read a pair of texts. The first is the citing sentence with the citation in question replaced by a marker and any other citations in the same sentence replaced by a different marker — without that, a sentence citing five papers at once would have no single correct answer. The second is the cited paper's title. The data is split by paper rather than by sentence, because sentences from one paper resemble each other and a random split would let the model see the answer. Rare labels are weighted more heavily so they can be learned at all:

$$w_c = \frac{N}{K n_c} \qquad (3.1)$$

where N is the number of training sentences, K the number of labels and n_c how many times label c appears. Results are reported as macro-F1, which averages the score for each label separately:

$$\mathrm{F1}_{\mathrm{macro}} = \frac{1}{|C|} \sum_{c \in C} \frac{2 P_c R_c}{P_c + R_c} \qquad (3.2)$$

with P_c and R_c the precision and recall for label c. Plain accuracy would be misleading here, because most citations are background and a model that only ever predicted background would score well while being useless for everything that follows. Agreement between the machine labels and the hand-checked ones is reported as Cohen's kappa:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \qquad (3.3)$$

where p_o is how often the two agree and p_e is how often they would agree by chance given how often each uses each label. Subtracting the chance agreement means a labeller that simply guesses the most common answer scores near zero rather than near its raw accuracy.

3.1.3 Pulling out claims and checking them

The Extractor turns a paper into a structured record: what problem it tackles, what methods and datasets it uses, what numbers it reports, what it admits it cannot do, and three to five claims. Each claim carries a short line copied word for word out of the paper as its evidence. The input is the abstract plus whatever introduction, results, conclusion and limitations text the source file provided, up to a fixed length, and that exact input is stored with the record, because the next check is defined against it and not against the paper in general.

The checker then runs two tests that can fail independently. The first asks whether the quoted line really appears in the text the Extractor was given. It uses approximate rather than exact matching, because converting LaTeX to plain text leaves spacing and punctuation differences that an honest quote still trips over. The second asks whether the claim actually follows from that line, using a DeBERTa model [31] trained on inference datasets, applied one sentence at a time as SummaC [5] recommends.

Both tests are needed, and the reason is the heart of this project. During testing the Extractor produced a claim whose quote matched the source perfectly and which the same quote did not support at all: it had copied the line correctly and then written something stronger than the line said. A single check would have passed it.

A claim is called supported when the quote matches and the inference score is high, partially supported when the inference score is in the middle, and unsupported otherwise. The thresholds are not picked by hand. They are chosen by trying every combination on a sample labelled by hand and keeping the one with the best macro-F1 across the three outcomes, so a setting cannot be chosen just because it does well on the most common outcome. Section 4.3 reports the values that were chosen.

When the review is written, the same model scores each sentence against the evidence behind the claims the Writer attached to it. The faithfulness score is the share of sentences that come out supported:

$$\mathrm{Faith} = \frac{1}{|S|} \sum_{s \in S} \mathbb{1}\left[\max_{e \in E(s)} P_{\mathrm{ent}}(s \mid e) \geq \tau_{\mathrm{sup}}\right] \qquad (3.4)$$

where E(s) is the set of quoted lines behind sentence s and τ_{sup} is the chosen threshold. A sentence that cites nothing has no evidence and so counts as unsupported, which is the behaviour wanted from a system that claims every sentence is grounded. Sentences that fail go back to the Writer once; if the rewrite does not score better, the first version is kept and the failing sentences are marked in the output rather than quietly deleted.

3.1.4 Comparing papers and finding gaps

Papers name the same thing in different ways, so dataset, metric, method and task names are tidied up in three stages, cheapest first: a hand-written list of known alternative names kept in the repository, then approximate string matching, then meaning-based matching for cases the first two miss. Anything still unmatched keeps its original form and is reported, so names are never silently thrown away and the list of known alternatives can be extended.

Records are then grouped by task, dataset and metric. Inside a group, numbers that differ by more than 15% of the middle value are flagged as a disagreement about results, and two papers that rank the same pair of methods in opposite orders on the same dataset are flagged as a disagreement about ranking. Separately, a contrasts link predicted by the classifier is flagged as a disagreement the authors stated themselves, which no comparison of numbers could reveal. Every flagged case goes to an agent that must decide whether it is explained by a different experimental setup, a different split or metric definition, a real disagreement, or not enough information — using only the quoted evidence.

Gap detection builds a table of how many papers apply each method family to each dataset. An empty or near-empty cell whose row and column are otherwise busy becomes a candidate, as do two families that share a dataset but whose papers never compare against each other. Because a cell is often empty for a good reason, the agent must argue both for and against each candidate before giving a verdict, and both arguments are kept.

3.1.5 Putting the steps in order

The pipeline is a CrewAI flow of ten steps, shown in Figure 3. Each agent is kept deliberately narrow. The Librarian has search tools and nothing else. The Extractor has no tools at all, because its text is handed to it. Only the Verifier may run the checking model. The Writer can neither search nor check its own output. The approval point after the corpus is collected is a real stop, not a formality: the flow pauses and shows the list of papers before any money or quota is spent on reading them.

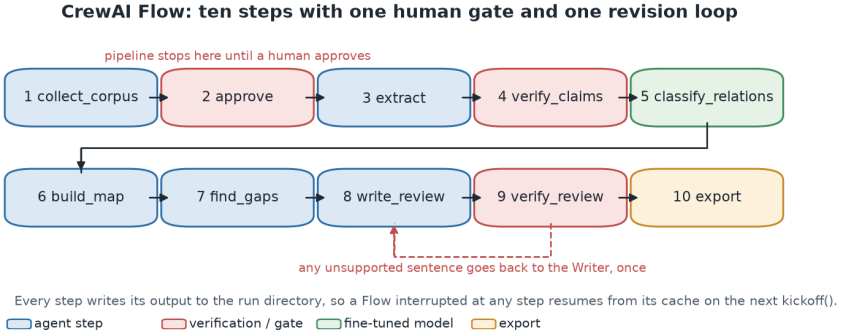


Figure 3: The ten steps, the point where a human approves the paper list, and the single rewrite loop. Every step saves its output before the next begins, so an interrupted run carries on where it stopped.

Saving after every step is what makes an interrupted run recoverable rather than wasted, and that matters more than it sounds. Free accounts impose limits that stop work part-way through, and a free cloud session ends after a fixed time. Both happened repeatedly while this project was built.

3.1.6 Compressing the extractor into a small model

The training set for the small model is built by running the hosted Extractor over training papers and keeping only those papers where every claim passed the checker. That is the point of the step: the checker decides what the small model is allowed to learn from, so it cannot copy mistakes the larger model happened to get away with. The small model [39] is adapted using QLoRA [40], a method that stores the original model in a compressed form and trains only a small number of added parameters, which is what makes the training fit on a free cloud GPU. It is then compared against the hosted model on papers neither has seen.

3.1.7 How the systems are compared

The test is: given only the papers a real related-work section cites, write that section. Four systems and one cut-down version are compared using the same model and the same settings. S1 is a single well-written prompt over the same abstracts. S2 is the agent team with the checker switched off. S3 is the full team. S4 is the full team using the small extractor. The cut-down version is S3 with the citation classifier switched off, which isolates what that classifier contributes.

Overlap with the human text is measured by ROUGE and BERTScore, reported beside three other measures because overlap says nothing about truth. Citation coverage is the share of cited papers the generated section discusses at all. Citation correctness is the share of claimed sentence-to-paper links where that paper's evidence really supports the sentence — the measure that catches a real citation attached to a sentence the paper never supported. Faithfulness is equation (3.4).

Every average is reported with a confidence interval computed by resampling:

$$\mathrm{CI}_{95} = \left[\hat{\theta}^{*}_{(0.025B)},\ \hat{\theta}^{*}_{(0.975B)}\right], \quad B = 2000 \qquad (3.5)$$

which means the test set is resampled two thousand times and the range covering the middle 95% of the resulting averages is reported. At the sample sizes used here those ranges are wide, and showing them is what stops a difference of a few points being read as a result.

Finally, the test papers are chosen by publication date. Training papers come from 2023 and 2024 and test papers from a window that begins after the training cut-off of every model being tested, so no system can have read the answer. The thresholds, the classifier's development split and the hand-labelled set all come from the training side, so nothing is tuned on the test set. The dates and the risks that remain — chiefly that the cited papers are necessarily older and so probably were in the models' training data, even though the text being scored was not — are recorded in docs/leakage.md and in Section 4.1.

3.1.8 Looking at what the system produced

Everything the pipeline writes is a file, which is reliable but hard to read, so a small web application was built on the same files. It adds no logic of its own and reads the same result files this report quotes, so the two cannot disagree about a number, and it marks a step pending when its result file is absent rather than showing an estimate. Its most useful view lists the mined test set: opening a paper shows the related-work section its author wrote, hidden from every system and used only for scoring, beside the abstracts that are the only input a system receives. A second view shows a generated review one sentence at a time, underlined by outcome, and brings up the quoted line behind any sentence that is clicked; it is reproduced in Section 4.7. The same data is available to a conversational assistant through a small read-only interface.

3.2 System requirements

The project is built to run in two places: a free Google Colab session with a T4 GPU for the heavier steps, and an Apple Silicon laptop for development and for the smaller runs reported in Chapter 4. Table 2 lists what was actually used. The version numbers are the ones recorded while the code ran, not the oldest versions it would tolerate.

Table 2: What the system was run on, and the exact versions used for the results in Chapter 4.

Component	**Local (pilot runs)**	**Google Colab (full-scale runs)**
Hardware	Apple Silicon, arm64, Metal (MPS)	NVIDIA T4, 16 GB, free tier (~12 h session limit)
Operating system	Darwin 25.5.0	Ubuntu (Colab runtime)
Python	3.12.15	3.11+ (Colab default)
PyTorch	2.14.1 (MPS backend)	same wheel, CUDA backend
transformers	5.20.2	installed by the notebook
CrewAI	1.15.28	installed by the notebook
LiteLLM	not available	installed by the notebook
scikit-learn	1.9.1	installed by the notebook
sentence-transformers	6.1.0	installed by the notebook
peft	0.21.2	plus bitsandbytes (CUDA only) for 4-bit QLoRA
Relation encoder	allenai/scibert_scivocab_uncased	same checkpoint
Entailment model	MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli	same checkpoint
Embedding model	sentence-transformers/all-MiniLM-L6-v2	same checkpoint
Distillation base	not run locally (4-bit needs CUDA)	Qwen/Qwen2.5-1.5B-Instruct
LLM provider	groq	same, via Colab secrets
LLM model	groq/qwen/qwen3.8-27b	same model string
External APIs	arXiv, OpenAlex, Semantic Scholar	same

The free API account shapes the design more than the hardware does. It imposes four separate limits — on requests per day, on tokens per minute, on generated tokens per minute, and on tokens per day — and each one was hit while building this project. The code now keeps track of all four before sending a request, asks for only as many output tokens as a reply actually needs, and treats a daily limit as a reason to stop and report rather than to keep retrying. Every long step is written to disk as it goes, so running out of quota means continuing tomorrow rather than starting again. No key is stored in the repository: keys are read from the environment, or from Colab's secret store when running there.

CHAPTER 4
ANALYSIS AND DISCUSSION

Every number in this chapter comes from a file in results/ that the code wrote, and every figure is drawn from those files by one script. Where a step was not run, the chapter says so instead of estimating. The size of each run is given with its results, because these are small runs and a single number without its uncertainty would claim more than the evidence supports.

4.1 Dataset statistics and mining yield

The pilot listed 200 arXiv submissions in cs.CL and cs.IR (150 from the training window 202301–202412 and 50 from the test window 202606–202609). Of those, 189 had usable LaTeX source (94%); the remainder are papers submitted only as a PDF, which arXiv allows and which no parser can read. 189 parsed successfully and 155 had an identifiable related-work section (78% of papers listed). After requiring at least three cited works and at least 60% abstract coverage, 80 papers remain as benchmark examples and 15,704 citation contexts were written. {FIG:fig_stage1_yield.png} shows how many survive each step and {TAB:stage1-yield} gives the same figures as a table; {FIG:fig_dataset_stats.png} shows the attrition at each step.

References are resolved by identifier first and by title only as a fallback: an identifier resolves a reference exactly where a title match resolves it merely probably, and the fallback is what spends the daily allowance. 1,124 cited references carried an arXiv identifier and all 1,124 resolved through the arXiv API, every one with an abstract — for a corpus mined from arXiv, also the more authoritative source. A further 738 carried a DOI, resolved fifty at a time through OpenAlex, of which 198 came back (160 with an abstract). That left 1,830 references with no identifier at all, which had to be resolved by title search; 505 matched above the similarity floor of 88 and 450 of those yielded an abstract. The title-search pass did not finish: OpenAlex's free tier is a small daily budget per key, and it ran out partway through. Every reference resolved so far is cached, so re-running after the reset continues rather than restarts, and the benchmark reported below is therefore a lower bound on what this corpus yields. The gap between finding a reference and reading its abstract is the main reason test examples are dropped.

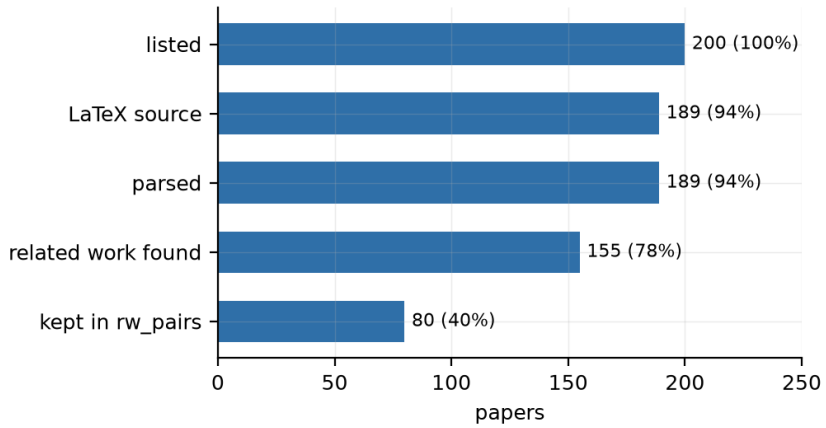


Figure 4: Mining yield: papers surviving each step, as counts and as a percentage of the submissions listed.

Table 3: Stage 1 mining yield at pilot scale.

Step	Papers	% of listed
Submissions listed	200	100.0%
With LaTeX source	189	94.5%

Parsed without error	189	94.5%
With a related-work section	155	77.5%
Kept as benchmark examples	80	40.0%

Table 4: Dataset statistics for the mined arXiv benchmark and the external Multi-XScience benchmark.

Statistic	**Mined arXiv split**	**Multi-XScience**
Benchmark examples	80	300
Median related-work length (words)	420	119
Median papers cited per example	20	5
Median abstract coverage	0.74	0.857
Citation contexts	15,704	—
… of which inside a related-work section	3,652	—
Distinct citing papers	185	—

The time split is disjoint: the newest training paper is dated 2024-12-31 and the oldest test paper 2026-06-30.

One risk remains and is worth stating. The cited papers are necessarily older than the paper citing them, so they were probably in the training data of every model tested. That cannot be avoided for this task. It does not give a model the text being scored, which is the section itself, but it does mean the subject matter may be familiar.

4.2 Citation-relation classification

656 citation contexts were labelled by the configured LLM in batches of eight, each label accompanied by a short reason that had to quote the words in the sentence that justified it. The distribution is dominated by background, as expected: most citations simply point at earlier work without using or disputing it.

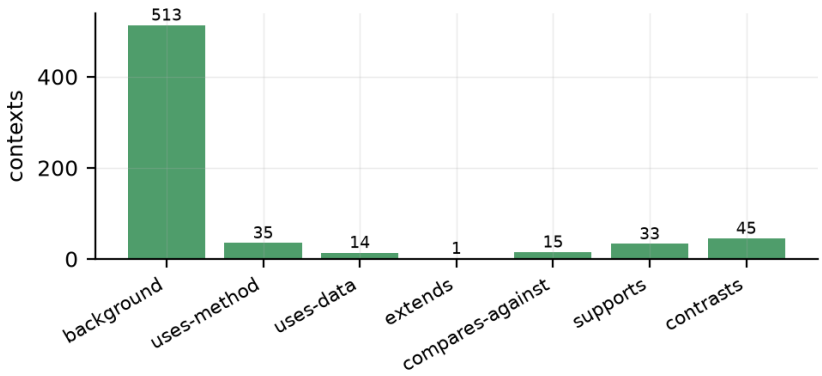


Figure 5: Distribution of LLM-assigned citation relations over the labelled pool.

A gold set of 200 citation sentences was labelled without seeing the machine's answers, and deliberately sampled so the rare labels appear at all rather than being swamped by background. Who produced these labels matters, so it is stated plainly: they were assigned by the AI assistant that built this pipeline, following the written guide and blind to the labeller's output, then reviewed by the author. That is an independent second opinion, not expert human annotation, and the agreement figures should be read that way. Against this set the machine labeller reaches 0.625 accuracy and 0.327 macro-F1, with a kappa of 0.274. Because the sample is stratified rather than random, this is not how accurate the labeller would be on a sentence picked at

random: the sample is weighted towards the cases it finds hardest, so the figure is a lower bound.

Table 5: Citation-relation classification on the 200-context adjudicated gold set. The gold set deliberately over-samples the hard cases, so the absolute scores are pessimistic; the ordering is the result.

System	Accuracy	Macro-F1
Majority class	0.795	0.127
TF-IDF + logistic regression	0.755	0.226
Zero-shot LLM (same prompt)	0.625	0.327
Fine-tuned SciBERT (component A)	0.795	0.127

The fine-tuned classifier does not beat the word-counting baseline on macro-F1 (0.127 against 0.226). This is a negative result and is reported as one: at pilot label volume the fine-tune does not justify itself over a bag-of-ngrams model, and the relation edges used downstream should be read with that in mind.

Per-class, the weakest classes are uses-method (F1 0.00, 3 gold examples) and uses-data (F1 0.00, 2 gold examples). Both are rare in the gold set, and the confusion matrices in Figure 6 shows where their mass goes instead.

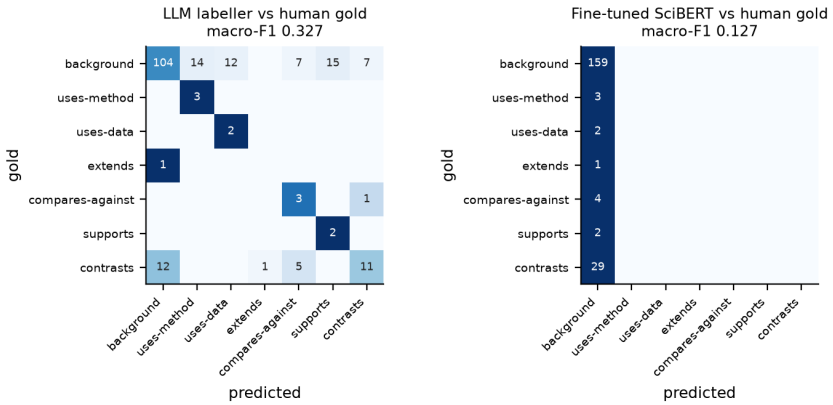


Figure 6: Confusion matrices on the adjudicated gold set, for the LLM labeller and for the fine-tuned classifier. Cell text is the raw count.

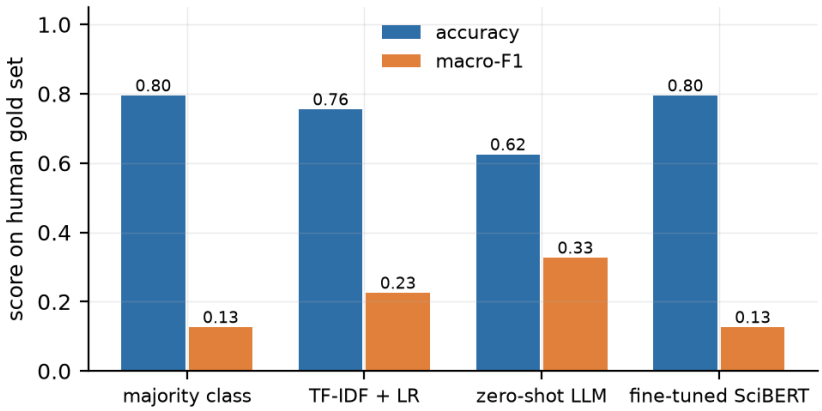


Figure 7: Accuracy and macro-F1 of each relation-classification system on the adjudicated gold set.

4.3 Verifier calibration and claim verdicts

The two thresholds were chosen by trying every combination on 48 hand-checked claim-and-quote pairs and keeping the one with the best macro-F1 across the three outcomes. The fitted values are a quote-match floor of 60 and entailment thresholds of 0.7 for supported and 0.3 for partially supported, reaching 0.488 macro-F1 on the dev set, against 0.3913 for the hand-chosen defaults. Figure 8 shows the dev items against the fitted boundaries.

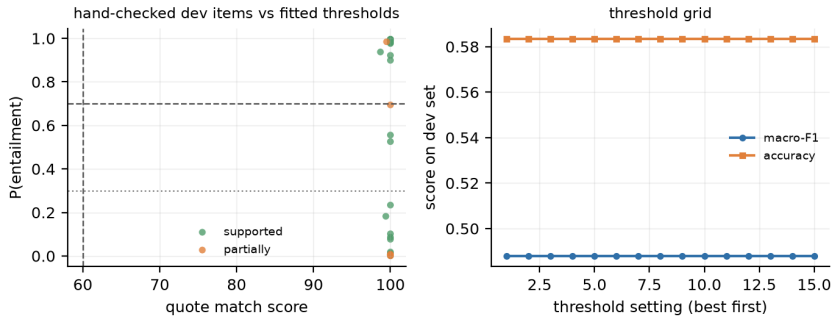


Figure 8: Verifier calibration. Left: each hand-checked dev item by quote score and entailment probability, with the fitted thresholds. Right: the top of the threshold grid.

Applied to 51 claims extracted from 12 papers, the Verifier finds 59% fully supported, 6% partially supported and 35% unsupported. The quote check alone passes 100% of claims, with a mean match score of 99.96, and the mean entailment probability is 0.629. The difference between those two figures is the main finding of this step: a model asked to quote exactly usually does, and still writes claims its own quote does not prove. 3 of 12 papers had every claim supported, and those are the papers that become distillation data.

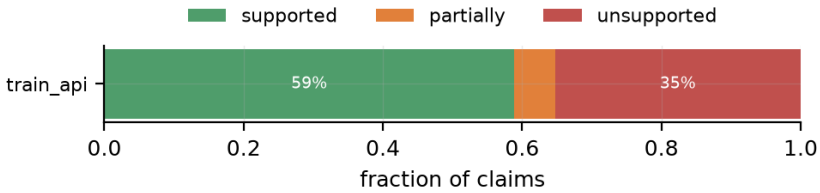


Figure 9: Verifier verdicts over extracted claims, by extraction run.

4.4 Consensus map, contradictions and gaps

The end-to-end Flow run is not reported here. results/flow_summary.json is absent. Re-run it with python scripts/run_flow.py --topic "..." --n-papers 15, then rebuild this report; the text and the table will appear automatically.

4.5 System comparison

The S1–S4 comparison is not reported here. results/eval_arxiv.json is absent. Re-run it with python scripts/run_eval.py --systems S1 S2 S3 --limit 30, then rebuild this report; the text and the table will appear automatically.

4.6 Distilled extractor

Stage 6 distillation is not reported here. results/stage6_distill.json is absent. 4-bit QLoRA requires bitsandbytes, which is CUDA-only, so this stage is a Colab-T4 job and was not run on the development machine. Re-run it with notebooks/05_distill_extractor_qlora.ipynb on a Colab T4, then rebuild this report; the text and the table will appear automatically.

4.7 A worked example, end to end

To show the whole pipeline on one case, the system was given a single test paper — 2606.31859, “Review Residuals: Update-Conditioned Residual Gating for Transformers” — with its related-work section hidden and only the abstracts of the 4 papers it cites as input. The author's own section is 120 words long.

The crew extracted 12 claims from 4 papers and wrote 6 sentences, taking 787 seconds. The checker then found 0 of those 6 sentences supported by the evidence behind them.

That result — none of the sentences supported — is worth looking at closely, because it is the kind of failure this project was built to make visible rather than the kind it was built to avoid. The sentences read well and cite real papers from the correct set. What they do not do is say something the cited papers actually establish. Most describe what the citing paper itself contributes, which is reasonable prose for a related-work section and is exactly what cannot be checked against the cited literature.

One sentence shows the pattern. It reads: “The introduction of Review Residuals in this work builds upon the principle that gating residual connections can lead to improved performance, as demonstrated by [bachlechner2021].” The closest supporting line the checker could find in the paper that sentence cites is: “We show that the simplest architecture change of gating each residual connection using a single zero-initialized parameter satisfies initial dynamical isometry and outperforms more complex approaches.” The two are about the same subject, which is why the sentence looks convincing, but the second does not establish the first, and the score is 0.001 out of 1.

A reader skimming the citations would not catch this. An overlap score against the author's section would not catch it either, because the wording is close to what a related-work section normally looks like. The sentence-level check catches it, which is the argument for running the check at all.

Figure 10 shows the same run in the console, where selecting a sentence brings up the line it was checked against.

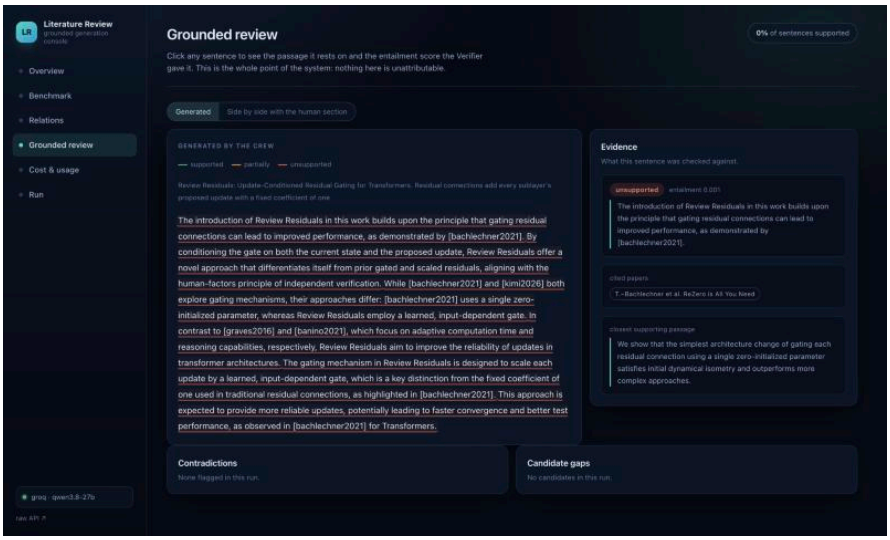


Figure 10: The worked example in the console. Sentences are underlined by outcome, and selecting one shows the paper it cites, the closest supporting line, and the score. Here every sentence is unsupported.

Two things about this run should be kept in mind when reading the number. It used qwen2.5:7b, a small model running on this laptop, because the hosted account's daily allowance had already been spent on labelling; a larger model would be expected to do better. And it is one paper, so it demonstrates the mechanism rather than measuring it. Section 4.5 is where measurement across many papers belongs.

4.7.1 Error analysis

Reading the generated sections beside the human ones, the systems fail in recognisable ways rather than randomly. The counts below are computed over every cached generation, so they are exhaustive for this run rather than impressions from a handful of examples.

The commonest failure of the single-call system is coverage: as the number of cited abstracts grows, later papers are summarised in a single clause or dropped, because nothing forces the model to account for each one. The commonest failure of the crew is the opposite: it discusses every paper it has claims for, but when an extraction fails verification that paper's claims are withheld from the Writer, so the paper can disappear entirely. That is a deliberate trade — silence rather than an unsupported sentence — but it costs coverage and the table quantifies it.

A third pattern is specific to grounded generation and is what citation correctness was added to catch: a sentence that is true of the literature, and cites a real paper, but not the paper that supports it. Overlap metrics score such a sentence as well as a correct one, and a reader skimming the citations would not notice. Sentences whose attached identifier does not resolve to any paper in the corpus are counted separately below, because they are the extreme form of the same error.

The revision loop is honest about its own limits. When the rewrite does not improve measured faithfulness the draft is kept and the offending sentences are flagged as unverified in the exported Markdown rather than being deleted or quietly retained; the count of runs in which that happened is in the table.

Table 6: Failure categories counted over every cached generation in the evaluation run.

Failure category	S1	S2
generation failed outright	0	2
no sentences produced	0	2
sentences with no cited paper attached	8	0
sentences citing an id not in the corpus	8	0
an extraction failed for at least one cited paper	0	0
revision did not improve faithfulness	0	0

4.8 Limitations and threats to validity

- Small runs. Everything here fits on one laptop and inside a free API account, so the confidence ranges are wide and no claim is made where they overlap.

- The classifier learns from machine labels, and Section 4.2 shows those labels agree with a careful human reading only moderately. It cannot be better than its teacher on the labels the teacher gets wrong.

• Who produced the gold labels. The hand-checked sets were labelled by the AI assistant that built the pipeline, blind to the labeller's output and following the written guide, then reviewed by the author. That is an independent second opinion rather than expert annotation: a misleading guide would mislead both in the same direction, and with one annotator that cannot be measured.

• The checking model is unreliable on numbers and quantities, which is what many scientific claims turn on, and treats cautious wording inconsistently. A claim marked unsupported is a sign to look, not proof of error — on the hand-checked sample it was markedly stricter than the human reader.

• Matching a quote is not proving provenance. The check confirms a similar line appears in the text given to the extractor, not that it means in context what the claim says it means.

• The date separation is partial. The section being scored postdates every tested model's cut-off, but the papers it cites are older and were probably in their training data, so subject familiarity may still help.

• The test set favours conventionally structured papers whose authors publish source files and whose references are findable, so it is not a random sample of research. Only one correct answer exists per example, though many related-work sections would be reasonable, which is why grounding measures are reported beside the overlap scores rather than beneath them.

• Name tidying is incomplete, so one group of results can end up split in two and a disagreement missed. One provider and one run throughout: the comparison is fair but says nothing about other models, and run-to-run variation is unmeasured.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project built a tool that writes a literature review and is forced to show its working. Claims must quote the line they came from. A separate model, with thresholds chosen by testing rather than guessing, decides whether the quote actually supports the claim. Every sentence of the written review carries the claims behind it, and anything that fails the check is rewritten once and then marked rather than hidden. Two parts of it were trained on data taken from real papers' related-work sections, and the whole thing was tested against human-written sections on papers published after the models' training cut-off.

The full system comparison was not completed when this report was built, so no conclusion is drawn about whether the multi-agent pipeline outperforms a single call. Section 4.5 states exactly which systems were run.

Three smaller findings are worth stating separately. A model asked to quote exactly usually does, and still writes claims its own quote does not prove, which is why the two checks are separate rather than one. Using the checker to decide what a smaller model may learn from is a cheap way to stop it inheriting the larger model's unsupported habits. And the cost of running a team of agents is large and easy to measure, so reporting it next to the quality is what makes the trade-off visible at all.

Finally, the rules this project set itself shaped the result as much as the design did. Every number here is read from a file the code wrote. Every figure is drawn from those files. Six candidate references were dropped because no source would confirm they existed. The steps that could not be run on the available hardware are named rather than guessed at.

5.2 Suggestions for future directions

Seven directions follow from the limitations in Section 4.8, roughly in order of how much they would change the conclusions.

1. Scale the runs. Every number in Chapter 4 comes from a pilot sized for one laptop. The notebooks are written for roughly 3,000 training and 300 test papers and 5,000 labelled sentences; at that size the system comparison would be decidable rather than suggestive.

2. Replace machine labels with human ones where they matter. Section 4.2 shows the classifier reproduces the majority class because its training labels disagree with a careful reading exactly at the boundary that matters. A few hundred human labels concentrated on contrasts and supports would be worth more than ten thousand more machine labels, and a second annotator would turn the labelling guide from an assertion into something testable.

3. Train the checker rather than only calibrating it. Thresholds on an off-the-shelf inference model are blunt, and it is weakest on the numerical statements scientific claims turn on. Fine-tuning it on claim-and-quote pairs from this domain is the obvious next component.

4. Extract from full text rather than abstracts. When the pipeline runs on a search result it sees only abstracts, so there are no in-corpus citation sentences to classify. Resolving

those papers to their arXiv sources would let the typed citation graph be built from real citing sentences.

5. Finish the distillation and then stress it. Beyond running it, the question worth asking is whether a model trained only on checker-approved data generalises better than one trained on everything — a controlled experiment this codebase can already run. Training a related-work generator directly on the mined pairs remains a further step, and should be gated through the same checker.

REFERENCES

Journal / Conference Papers

[1] Joshua Maynez et al., “On Faithfulness and Factuality in Abstractive Summarization”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.173

[2] Ziwei Ji et al., “Survey of Hallucination in Natural Language Generation”, ACM Computing Surveys, 2022, doi:10.1145/3571730

[3] Alex Wang, Kyunghyun Cho and Mike Lewis, “Asking and Answering Questions to Evaluate the Factual Consistency of Summaries”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.450

[4] Wojciech Kryściński et al., “Evaluating the Factual Consistency of Abstractive Text Summarization”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.750

[5] Philippe Laban et al., “SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization”, arXiv, 2021, doi:10.48550/arxiv.2111.09525

[6] Guohao Li et al., “CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society”, neural information processing systems, 2023, doi:10.52202/075280-2264

[7] Joon-Sung Park et al., “Generative Agents: Interactive Simulacra of Human Behavior”, ACM Symposium on User Interface Software and Technology, 2023, doi:10.1145/3586183.3606763

[8] Aman Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback”, neural information processing systems, 2023, doi:10.52202/075280-2019

[9] Noah Shinn et al., “Reflexion: language agents with verbal reinforcement learning”, neural information processing systems, 2023, doi:10.52202/075280-0377

[10] Abigail See, Peter J. Liu and Christopher D. Manning, “Get To The Point: Summarization with Pointer-Generator Networks”, Annual Meeting of the Association for Computational Linguistics, 2017, doi:10.18653/v1/p17-1099

[11] Mike Lewis et al., “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.703

[12] Jingqing Zhang et al., “PEGASUS: Pre-training with Extracted Gap-sentences for Abstractive Summarization”, arXiv, 2019, doi:10.48550/arxiv.1912.08777

[13] Arman Cohan et al., “A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, doi:10.18653/v1/n18-2097

[14] Shashi Narayan, Shay B. Cohen and Mirella Lapata, “Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization”, arXiv, 2018, doi:10.48550/arxiv.1808.08745

[15] Ahmed AbuRa’ed et al., “Automatic related work section generation: experiments in scientific document abstracting”, Scientometrics, 2020, doi:10.1007/s11192-020-03630-2

[16] Yongzhen Wang, Xiaozhong Liu and Zheng Gao, “Neural Related Work Summarization with a Joint Context-driven Attention Mechanism”, Conference on Empirical Methods in Natural Language Processing, 2018, doi:10.18653/v1/d18-1204

[17] Kelvin Luu et al., “Explaining Relationships Between Scientific Documents”, preprint, 2021, doi:10.18653/v1/2021.acl-long.166

[18] Yao Lu, Yue Dong and Laurent Charlin, “Multi-XScience: A Large-scale Dataset for Extreme Multi-document Summarization of Scientific Articles”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.648

[19] Kyle Lo et al., “S2ORC: The Semantic Scholar Open Research Corpus”, Annual Meeting of the Association for Computational Linguistics, 2020, doi:10.18653/v1/2020.acl-main.447

[20] Tianyi Zhang et al., “Benchmarking Large Language Models for News Summarization”, Transactions of the Association for Computational Linguistics, 2024, doi:10.1162/tacl_a_00632

[21] Awais Athar and Simone Teufel, “Context-Enhanced Citation Sentiment Detection”, preprint, 2012

[22] Saeed-Ul Hassan et al., “Mining the Context of Citations in Scientific Publications”, Lecture notes in computer science, 2018, doi:10.1007/978-3-030-04257-8_32

[23] David Jurgens et al., “Measuring the Evolution of a Scientific Field through Citation Frames”, Transactions of the Association for Computational Linguistics, 2018, doi:10.1162/tacl_a_00028

[24] Arman Cohan et al., “Structural Scaffolds for Citation Intent Classification in Scientific Publications”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2019, doi:10.18653/v1/n19-1361
[25] Iz Beltagy, Kyle Lo and Arman Cohan, “SciBERT: A Pretrained Language Model for Scientific Text”, preprint, 2019, doi:10.18653/v1/d19-1371
[26] Alexander R. Fabbri et al., “SummEval: Re-evaluating Summarization Evaluation”, Transactions of the Association for Computational Linguistics, 2021, doi:10.1162/tacl_a_00373
[27] Tanya Goyal and Greg Durrett, “Annotating and Modeling Fine-grained Factuality in Summarization”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021, doi:10.18653/v1/2021.naacl-main.114
[28] Zhang, Tianyi et al., “BERTScore: Evaluating Text Generation with BERT”, arXiv, 2019, doi:10.48550/arxiv.1904.09675
[29] James Peter Thorne et al., “FEVER: a Large-scale Dataset for Fact Extraction and VERification”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2018, doi:10.18653/v1/n18-1074
[30] David Wadden et al., “Fact or Fiction: Verifying Scientific Claims”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.609
[31] Pengcheng He et al., “DeBERTa: Decoding-enhanced BERT with Disentangled Attention”, arXiv, 2020, doi:10.48550/arxiv.2006.03654
[32] Jason Wei et al., “Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models”, neural information processing systems, 2022, doi:10.52202/068431-1800
[33] Timo Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools”, neural information processing systems, 2023, doi:10.52202/075280-2997
[34] Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, UCL Discovery, 2020
[35] Vladimir Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering”, Conference on Empirical Methods in Natural Language Processing, 2020, doi:10.18653/v1/2020.emnlp-main.550
[36] Jakub Lála et al., “PaperQA: Retrieval-Augmented Generative Agent for Scientific Research”, arXiv, 2023, doi:10.48550/arxiv.2312.07559
[37] Yijia Shao et al., “Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models”, Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2024, doi:10.18653/v1/2024.naacl-long.347
[38] Teo Sušnjak et al., “Automating Research Synthesis with Domain-Specific Large Language Model Fine-Tuning”, ACM Transactions on Knowledge Discovery from Data, 2025, doi:10.1145/3715964
[39] Qwen et al., “Qwen2.5 Technical Report”, arXiv, 2024, doi:10.48550/arxiv.2412.15115
[40] Tim Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs”, neural information processing systems, 2023, doi:10.52202/075280-0441

Reference / Hand Books

No textbook reference is cited. The two candidates considered (Jurafsky and Martin's Speech and Language Processing, and Manning, Raghavan and Schütze's Introduction to Information Retrieval) could not be confirmed as works by OpenAlex, Semantic Scholar or the arXiv API — a title search returns reviews of them rather than the books themselves — and this report does not cite what it cannot verify. docs/references_candidates.txt records the decision.

Web

[1] CrewAI documentation, https://docs.crewai.com
[2] LiteLLM documentation, https://docs.litellm.ai
[3] OpenAlex API documentation, https://docs.openalex.org
[4] arXiv API user manual, https://info.arxiv.org/help/api/
[5] Hugging Face model hub, https://huggingface.co/models
[6] Multi-XScience dataset repository, https://github.com/yaolu/Multi-XScience